TalkLate

Traduccion en tiempo real con conciencia cultural y emocional

19. Seguridad con Trivy

Trabajo de grado

Julian David Diaz Medina

Diego Alejandro Franco Atehortua

Director: Wider Farid Sanchez Garzon

Version 1.1 - Noviembre de 2026

Version: 1.0 (05/11/2026)

TalkLate escanea su codigo con Trivy en cada pull request, en cada push a `develop` y `main`, y cada lunes. El escaneo busca dependencias con vulnerabilidades conocidas, secretos escritos en el codigo y configuraciones inseguras. Si encuentra algo de severidad alta o critica, el CI falla y no se puede hacer merge.

1. Resultado actual

Que se escanea	Hallazgos HIGH / CRITICAL antes	Despues	Como se resolvio
Dependencias del backend (`backend/package-lock.json`)	1 HIGH (`body-parser` 1.20.8)	0	Actualizacion de `express` a 4.22.3, que trae `body-parser` 1.20.9
Dependencias del frontend (`frontend/package-lock.json`), de produccion	1 CRITICAL y 28 HIGH	0 (1 excepcion documentada)	`overrides` de npm hacia versiones parchadas, sin cambiar de version mayor (ver tabla 2)
Dependencias de desarrollo del frontend (eslint, TypeScript y sus herramientas)	22 HIGH	0	`overrides` de npm hacia versiones parchadas de la misma version mayor
Dependencias de las pruebas (`e2e/package-lock.json`)	0	0	-
Secretos en el codigo (claves, tokens, contrasenas)	0	0	Las credenciales viven en `.env`, que no se sube al repositorio
Configuraciones inseguras (IaC)	0	0	El proyecto no tiene Dockerfile ni infraestructura como codigo todavia; el escaneo queda activo para cuando la haya

2. Dependencias corregidas en el frontend

Todas son dependencias indirectas de las herramientas de Expo y React Native. Se fuerzan con `overrides` en `frontend/package.json` a la version parchada mas cercana, sin saltar de version mayor, para no romper nada. Despues del cambio pasan las pruebas, el chequeo de tipos y la exportacion web.

Paquete	Version antes	Version ahora	Severidad mas alta	Riesgo
`shell-quote` (react-devtools)	1.8.3	1.12.0	CRITICAL	Ejecucion de comandos por texto sin escapar
`@xmldom/xmldom` (herramientas de Expo)	0.8.10	0.8.15	HIGH	Inyeccion y denegacion de servicio con XML manipulado
`node-forge` (firma de codigo de Expo)	1.4.0	1.4.3	HIGH	Falsificacion de firmas y denegacion de servicio en TLS

Paquete	Version antes	Version ahora	Severidad mas alta	Riesgo
`js-yaml`	4.1.0	4.3.2	HIGH	Denegacion de servicio con YAML manipulado
`browserslist`	4.26.2	4.29.5	HIGH	Contaminacion de prototipos y denegacion de servicio
`picomatch` 2.x	2.3.1	2.3.2	HIGH	Denegacion de servicio por expresiones regulares
`ws` 7.x (react-devtools)	7.5.10	7.5.13	HIGH	Denegacion de servicio por agotamiento de memoria
`minimatch` 3.x y 9.x (desarrollo)	3.1.2 / 9.0.5	3.1.4 / 9.0.7	HIGH	Denegacion de servicio por patrones de busqueda
`brace-expansion` 1.x y 2.x (desarrollo)	1.1.11 / 2.0.1	1.1.20 / 2.1.6	HIGH	Denegacion de servicio por expansion de llaves
`glob` 10.x (desarrollo)	10.4.5	10.5.0	HIGH	Inyeccion de comandos en su linea de comandos
`flatted` (desarrollo)	3.3.3	3.4.2	HIGH	Contaminacion de prototipos al leer datos

3. Excepcion aceptada

Hallazgo	Por que no se corrige hoy	Riesgo real	Vence
CVE-2026-118664 en `decode-uri-component` 0.2.2 (via `expo-router` -> `query-string` 7)	La unica version parchada (0.5.1) es un modulo ESM que `query-string` 7 no puede cargar; forzarla rompe la navegacion de la app	Bajo: es una denegacion de servicio en el navegador o el celular de quien abre un enlace manipulado a la propia app; no expone el servidor ni datos	28/02/2027

La excepcion esta en `.trivyignore` con fecha de vencimiento: ese dia Trivy vuelve a fallar y el equipo revisa si `expo-router` ya trae una version compatible.

4. Como funciona

Archivo	Para que sirve
`trivy.yaml`	Configuracion: escaneres (`vuln`, `secret`, `misconfig`), tambien dependencias de desarrollo, severidades que bloquean (HIGH y CRITICAL), carpetas excluidas (`node_modules`, la web exportada y el template de UI original, que no se despliega)
`.trivyignore`	Excepciones justificadas, cada una con razon y fecha de vencimiento
`.github/workflows/seguridad.yml`	Instala Trivy, genera el reporte completo (todas las severidades) como artefacto y corre el escaneo que bloquea

Instalacion segura de Trivy en el CI. El workflow descarga el binario oficial de Trivy v0.69.3 y verifica su checksum SHA-256 antes de usarlo. No se usan las acciones `aquasecurity/trivy-action` ni `setup-trivy`: en marzo de 2026 sus etiquetas fueron comprometidas en un ataque a la cadena de suministro, y una herramienta de seguridad comprometida es justo lo que no se puede permitir. Fijar version y checksum evita ese riesgo.

Por que cada lunes. Cada semana se publican vulnerabilidades nuevas sobre paquetes que no cambiaron. El escaneo programado avisa aunque nadie haya tocado el codigo.

5. Como correrlo en local

```
# Instalar (Linux; en Mac: brew install trivy)
curl -sSfL -o trivy.tgz
https://github.com/aquasecurity/trivy/releases/download/v0.69.3/trivy_0.69.3_Linux-
64bit.tar.gz
echo "1816b632dfe529869c740c0913e36bd1629cb7688bd5634f4a858c1d57c88b75  trivy.tgz" |
sha256sum -c -
tar xzf trivy.tgz trivy

# Desde la raiz del repositorio
./trivy fs --config trivy.yaml .                                    # el mismo escaneo que
bloquea en CI
./trivy fs --config trivy.yaml --severity LOW,MEDIUM,HIGH,CRITICAL --exit-code 0 .   #
reporte completo
```

6. Que hacer cuando Trivy falla

1. Leer el reporte: paquete, version instalada y version con el parche.
2. Si es una dependencia directa, actualizarla (`npm update <paquete>` o subir la version en `package.json`).
3. Si es indirecta, agregar un `override` en `package.json` hacia la version parchada de la misma version mayor y correr las pruebas.
4. Si no hay parche compatible, documentar la excepcion en `.trivyignore` con la razon, el riesgo real y una fecha de vencimiento de no mas de 4 meses.
5. Un secreto encontrado nunca se ignora: se revoca la clave, se cambia y se saca del historial.

7. Relacion con la arquitectura

- Responde a DRV-03 (privacidad de los datos) y a la convencion de seguridad del proyecto: dependencias sin vulnerabilidades conocidas y ningun secreto en el codigo.
- Complementa los controles que ya prueba la piramide: cifrado AES-256-GCM, logs sin datos personales, limites de uso, JWT corto con refresco rotativo (ver *documento 18 (Piramide de pruebas)*).
- Decision registrada en ADR-018 (*documento 11 (Decisiones de arquitectura (ADR))*).